# Crittografia con OpenSSL – Applicazioni

Laboratorio del corso "Sicurezza dei sistemi informativi" (01UDUOV)
Politecnico di Torino – AA 2023/24
Prof. Cataldo Basile

*preparata da:*
Gabriele Gatti (gabriele.gatti@polito.it)

v. 3.8 (14/11/2023)

## Indice

**1 Applicazioni della crittografia: integrità** **6**

1.1 Autenticazione di messaggio (Message Authentication Code) . . . . . 6

1.1.1 Creazione manuale di un keyed-digest . . . . . 6

1.1.2 Creazione di un HMAC con OpenSSL . . . . . 7

1.2 Authenticated Encryption with Associated Data (AEAD) . . . . . 7

1.3 Firma digitale . . . . . 9

**2 Applicazioni della crittografia asimmetrica: chiavi di messaggio** **11**

**3 Infrastrutture a chiave pubblica** **13**

3.1 La Certification Authority (CA) . . . . . 13

3.2 Richiesta ed emissione di certificati digitali . . . . . 14

3.3 Revoca di certificati . . . . . 15

**4 Approfondimenti ed esercizi opzionali** **17**

4.1 Autenticazione . . . . . 17

4.2 Validazione di messaggi firmati . . . . . 18

4.3 Progettazione di protocolli . . . . . 21

4.3.1 Protezione di messaggio . . . . . 22

4.3.2 Protezione di canale . . . . . 22

4.4 Formati PKCS#7 . . . . . 24

## Scopo dell'esercitazione

Scopo dell'esercitazione è applicare le procedure per il calcolo delle primitive crittografiche viste nella precedente esercitazione. Anche questa esercitazione è basata sull'uso estensivo di OpenSSL (http://www.openssl.org/), una libreria crittografica rilasciata come software open-source con licenza Apache e disponibile su un ampio insieme di piattaforme, incluse Linux e Windows.

L'esercitazione è basata sull'uso di OpenSSL da linea di comando che permette di invocare varie funzioni crittografiche attraverso la shell digitando:

```
openssl command [ command_opts ] [ command_args ]
```

Per il corretto svolgimento di questa esercitazione sarà necessario usare i comandi:

```
dgst rand genrsa rsa pkeyutl enc
```

che sono già stati introdotti nell'esercitazione di laboratorio 2. Inoltre saranno utilizzati anche i comandi:

```
req ca verify crl x509
```

presentati di seguito. Per la lista completa delle opzioni ed una descrizione dettagliata dei comandi consigliamo di visionare le relative pagine `man`.

I file ed i programmi necessari allo svolgimento dei singoli esercizi sono disponibili scaricando l'archivio `03lab-materiale.zip`.

Per decomprimerli potete usare il seguente comando:

```
7z x "03lab - materiale.zip"
```

## openssl req

Il comando OpenSSL `req` serve a creare una richiesta di certificato X.509 in formato PKCS#10. La sintassi di `req` è la seguente:

```
openssl req [-in file] [-out reqfile] [-newkey arg] [-key file_chiave] [-text]
    [-pubkey] [-noout] [-keyout file_chiave]
```

in cui:

`-in` *file*, legge la richiesta dal file (serve solo se non viene usata l'opzione `-newkey`);

`-out` *reqfile*, salva nel file la richiesta;

`-newkey` *arg*, permette di creare una nuova chiave privata e la richiesta di certificato. Il parametro `arg` può essere espresso in due modi: (1) `rsa:nbits` genera una chiave RSA il cui modulo è lungo `nbits` bit; (2) `dsa:filename` genera una chiave DSA usando i parametri passati nel file `filename`;

`-key` *file_chiave*, crea una richiesta a partire da un chiave privata esistente e memorizzata nel file `file_chiave`;

`-pubkey`, estrae la chiave pubblica e la produce in output;

`-keyout` *file_chiave*, salva nel file la chiave privata generata;

`-text`, stampa la richiesta in forma testuale;

`-noout`, non stampa la versione codificata della richiesta (utile con `-text`).

## CA.pl

Per creare e gestire una semplice Certification Authority (CA), si può usare il comando `CA.pl`, distribuito sempre con OpenSSL. `CA.pl` è uno script Perl che permette di gestire da riga di comando le principali funzionalità di una CA. La sintassi è la seguente:

```
CA.pl [-newca]
```

Con l'opzione `-newca` si può creare un'intera "CA hierarchy". Il comando vi chiederà poi tramite prompt dei comandi tutti i dettagli della nuova CA. Tutti i file principali vengono creati nella cartella "`demoCA`". Per maggiori informazioni su `CA.pl`, potete leggere il `man`:

```
man CA.pl
```

## openssl ca

Per emettere o revocare un certificato X.509, si può usare il comando OpenSSL `ca`, il quale implementa una CA minimale in grado di firmare richieste di certificato (espresse secondo diversi formati) e generare le CRL. Inoltre `ca` mantiene un database testuale dei certificati emessi e del loro stato. La sintassi è la seguente:

```
openssl ca [-in file] [-out file] [-revoke file] [-gencrl]
```

in cui:

`-in` *file*, indica che il file *file* contiene la richiesta da firmare;

`-out` *file*, salva il certificato generato nel file *file*;

`-revoke` *file*, indica che il file *file* contiene il certificato da revocare. È importante notare che una CA generata con `CA.pl` memorizza tutti i certificati emessi nella cartella `newcerts` con nome `<serial_number>.pem`. Quindi per revocare un certificato è importante solo conoscere il serial number o ricavarlo tramite i dati dell'utente memorizzati nei certificati in `newcerts`. Inoltre l'opzione `-revoke` aggiorna lo stato del file `index.txt`;

`-gencrl`, indica che deve essere prodotta la CRL (sulla base dei dati in `index.txt`).

## openssl verify

Il comando OpenSSL `verify` permette di verificare un certificato e tutta la sua catena di certificazione. L'operazione di verifica consiste in una serie di operazioni separate:

1. Viene prima costruita la catena di certificati fino alla "root CA" (cioè il primo certificato *self signed* che trova) usando i nomi (DN) degli issuer. Viene inoltre verificato che l'issuer possa emettere certificati (estensione `KeyUsage`).
2. La seconda operazione consiste in una verifica che il "*purpose*", lo scopo per cui viene usato il certificato, sia corretto (guardate la sezione CERTIFICATE EXTENSIONS di `man x509`).
3. La terza operazione consiste nel verificare che la CA sia fidata.
4. Infine, vengono verificate le "validity" di ogni certificato (`notBefore` e `notAfter`) e le firme digitali.

Il certificato è considerato valido solo se tutte le operazioni vengono completate con successo.

Notate che, se una verifica fallisce, l'output è spesso criptico, ad esempio:

```
server.pem:  /C=AU/O=CryptSoft Pty Ltd/CN=Test CA (1024 bit)
error 24 at 1 depth lookup:invalid CA certificate
```

Il messaggio contiene il nome del certificato da verificare ed il subject, il codice di errore ed il numero di certificati (`depth`) esaminati prima che si verificasse l'errore (0 è il certificato stesso, 1 la sua CA, ...). La sintassi è la seguente:

```
openssl verify [-CAfile file] [-verbose] [certificates]
```

in cui:

`-CAfile` *file* indica che il file *file* contiene la lista di tutti i certificati fidati sotto forma di concatenazione (*append*) di certificati in formato PEM;

`-verbose` stampa informazioni aggiuntive sulle operazioni di verifica;

`certificates` è la lista di uno o più certificati da verificare (tutti in formato PEM).

## openssl crl

Il comando OpenSSL `crl` permette manipolare una Certificate Revocation List (CRL).
La sintassi è la seguente:

```
openssl crl [-in file] [-CAfile file] [-text] [-noout]
```

in cui:

`-in` *file*, indica che il file *file* contiene la CRL;

`-CAfile` *file* indica che il file *file* contiene i certificati usati per verificare la genuinità della CRL;

`-text`, stampa la richiesta in forma testuale;

`-noout`, non stampa la versione codificata della richiesta (utile con `-text`).

## openssl x509

Per firmare ed estrarre in formato testuale informazioni sui certificati X.509 si può usare il comando OpenSSL `x509`. In particolare con `x509` si possono visualizzare le informazioni relative ad un certificato e convertire certificati in diversi formati. Si possono inoltre modificare i "trust settings" dei certificati (si comporta cioè come una "mini CA"). La sintassi è la seguente:

```
openssl x509 [-inform DER|PEM] [-outform DER|PEM] [-in file] [-out file]
     [-noout] [-req] [-text]
```

in cui:

`-inform DER|PEM` specifica il formato di input in cui PEM è la codifica Base64 del certificato DER;

`-outform DER|PEM` specifica il formato di output;

`-in` *file* indica di leggere il certificato dal file specificato;

`-out` *file* salva il certificato in nel file specificato;

`-noout` indica di non produrre in output la rappresentazione Base64 del certificato;

`-req` indica che il file *file* passato mediante l'opzione `-in` contiene una richiesta e non un certificato (come si aspetterebbe di default);

`-text` visualizza tutti i dettagli del certificato in formato testuale.

# 1 Applicazioni della crittografia: integrità

## 1.1 Autenticazione di messaggio (Message Authentication Code)

### 1.1.1 Creazione manuale di un keyed-digest

Supponiamo che Alice voglia generare il keyed-digest di un messaggio che vuole inviare a Bob. Alice crea un file chiamato `msg` con un messaggio di testo tipo il seguente:

```
Questo e' un messaggio di prova per la creazione di un keyed-digest!
```

e crea un file `key` in cui memorizza la chiave che utilizzerà per calcolare il keyed-digest.

Come può Alice usare il comando `dgst` di OpenSSL per calcolare un keyed-digest su `msg`? Cosa deve conoscere Bob per verificarlo? Quali passi deve compiere Bob per verificarlo?

NOTA

Non serve definire formati avanzati o avere massima fedeltà agli standard per questa esercitazione che ha il fine di chiarire i concetti imparati a lezione. Per esempio, potete usare il comando `cat` della shell di Linux e le redirezioni di output di Linux ($>$ e $>>$) per simulare la concatenazione di dati.

**Soluzione**

Soluzione semplice: concateno messaggio e chiave e calcolo l'hash `hash( M | K )`.

```
echo 012345 >key
cat msg >>data
cat key >>data
```

Soluzione più efficace `hash( K | M | K )`

```
cat key >>data
cat msg >>data
cat key >>data
```

Il comando per generare il digest è:

```
openssl dgst [-algoritmo di digest] [-out output file] data
```

Bob per verificarlo deve:

- ricalcolare il digest usando il messaggio ricevuto `msg` e il segreto condiviso `key`
- confrontare il digest calcolato con quello ricevuto

Si può anche usare `hash( K | hash( K | M ) )`. Anche se non è conveniente dal punto di vista computazionale perché richiede due operazioni crittografiche, è ritenuto molto più sicuro (confrontate l'HMAC calcolato con `dgst` sotto).

Descrivete due possibili applicazioni della funzione di keyed-digest costruita sopra.

**Soluzione**

Il keyed-digest permette di garantire integrità dei messaggi e autenticazione dei dati. Può essere usato per la protezione di pacchetti scambiati con protocolli di comunicazione di rete e protezione dei digest di file come visto nell'esercitazione precedente.

### 1.1.2 Creazione di un HMAC con OpenSSL

Cercate adesso i comandi OpenSSL che permettano di calcolare direttamente un keyed-digest (creando il file `msg.hmac`) che usi SHA256 come funzione di hash.

**Soluzione**

```
openssl dgst -sha256 -binary -hmac chiave -out msg.hmac msg
```

Come possiamo verificare l'HMAC `msg.hmac` generato con il precedente comando OpenSSL? (Per confrontare due file binary potete usare `cmp file1 file2`)

**Soluzione**

```
openssl dgst -sha256 -binary -hmac chiave -out msg.hmac2 msg
cmp msg.hmac msg.hmac2
```

Proviamo adesso a verificare che l'HMAC è effettivamente in grado di rilevare le modifiche al messaggio. Cambiate anche un solo carattere nel contenuto del messaggio `msg` e ricalcolate l'HMAC. Cosa notate?

**Soluzione**

```
printf 'q' | dd of=msg bs=1 count=1 conv=notrunc
openssl dgst -sha256 -binary -hmac chiave -out msg.hmac3 msg
cmp msg.hmac msg.hmac3
```

L'HMAC generato dopo la modifica del file è completamente diverso dal primo HMAC, facendone il `cmp` è possibile identificare le modifiche.

Supponiamo che un attaccante abbia ottenuto l'accesso al contenuto del messaggio `msg`, ad esempio intercettando una comunicazione tra Alice e Bob. Sarebbe possibile per l'attaccante modificare il messaggio `msg` e rigenerare l'HMAC in modo da non rendere rilevabile la modifica del messaggio (in modo simile ai digest calcolati con `hashdeep` nell'esercitazione precedente per l'esercizio opzionale 4.3)?

**Soluzione**

No, gli servirebbe anche la chiave che in teoria dovrebbero conoscere solo Alice e Bob.

## 1.2 Authenticated Encryption with Associated Data (AEAD)

Adesso proviamo ad usare i metodi di authenticated encryption. Guardate gli algoritmi crittografici proposti da OpenSSL `enc` invocando il seguente comando:

```
openssl list -cipher-algorithms
```

Riconoscete qualche algoritmo che vi faccia pensare all'uso di AEAD?

**Soluzione**

Ci sono varie modalità di AES (gli `id-aes*-CCM` e `id-aes*-GCM`) e `ChaCha20-Poly1305`.

Provate ad usare gli algoritmi che avete identificato invocandoli da riga di comando. Avete avuto dei problemi?

**Soluzione**

OpenSSL non permette di usarli da riga di comando.

Purtroppo, come avrete notato, OpenSSL non permette di invocare da riga di comando primitive crittografiche di tipo AEAD (forse perché dovrebbero adattare l'API stabile ormai da molto tempo). Tali primitive crittografiche sono solo accessibili usando l'interfaccia di programmazione OpenSSL EVP.

Per permettervi di testare AEAD, abbiamo scritto uno script in Python che implementa AEAD con AES128 in GCM e che usa la seguente sintassi:

```
aes-gcm.py [-d|-e] plaintext associated_data ciphertext tag -K key -iv initialization_vector
```

In pratica, questo script accetta sempre in input quattro file. Se viene scelta l'opzione `-e` (encryption), lo script applica confidenzialità al file `plaintext` e scrive il file cifrato in `ciphertext` e calcola un tag di integrità per `plaintext` e `associated_data`, che scrive nel file `tag`. Se viene scelta l'opzione `-d` (decryption), lo script decifra i dati in `ciphertext` e li scrive nel file `plaintext` e verifica il tag di integrità che trova nel file `tag` usando `associated_data` e quello che decifra da `ciphertext`.

Più precisamente, il significato dei singoli parametri è il seguente:

- `[-e|-d]` permettono di scegliere una delle due possibilità, encryption e tag generation oppure decryption e tag verification come spiegato in precedenza;
- *plaintext* è il nome del file contenente i dati da cifrare (opzione `-e`) o il file in cui sarà salvato il risultato della decifratura (opzione `-d`);
- *associated_data* è il nome del file contenente di "dati associati" cioè usare solo nel calcolo/verifica del tag di integrità;
- *ciphertext* è il nome del file in cui sarà salvato il contenuto cifrato (opzione `-e`) o da cui sarà letto il file cifrato (opzione `-d`);
- *tag* è il nome del file in cui sarà salvato il tag di autenticazione;
- `-K` *chiave*, dove `key` è una stringa di 32 caratteri esadecimali che rappresenta la chiave da usare per la cifratura ed il calcolo del tag;

  **ATTENZIONE**

  L'opzione `-K` (maiuscola) non accetta un nome di file, inoltre la chiave deve essere di lunghezza corretta. Inoltre, verrà visualizzato uno "strano" errore (alla funzione `fromhex()`) se usate un numero dispari di cifre esadecimali.

- `-iv` *vettore* è una stringa di cifre esadecimali che rappresenta il vettore di inizializzazione.

`aes-gcm.py` è disponibile nell'archivio del materiale associato a questa esercitazione di laboratorio. Dopo averlo decompresso, ricordate di renderlo eseguibile con il comando `chmod +x aes-gcm.py`.

Proviamo ora ad usare lo script `aes-gcm.py`. Alice crea due file:

- `plain` in cui Alice salverà il messaggio da mantenere confidenziale, es. "*Bob, questo è un messaggio molto molto segreto*";
- `aad` in cui salverà il contenuto da usare solo per il MAC, es. "*Messaggio segreto da Alice a Bob*";

Poi Alice calcola il tag ed il file cifrato con AES in GCM e li salva nei file `cipher` e `tag`, rispettivamente. Scrivete il comando per farlo.

**Soluzione**

Il comando da eseguire è il seguente:

```
aes-gcm.py -e plain aad cipher tag -K chiave -iv iv
```

Proviamo quindi a verificare come le proprietà dell'AEAD di garantire autenticità possono essere usate. Per farlo provate ad alterare il messaggio da cifrare e ricalcolare tag e messaggio cifrato (es. in `cipher2` e `tag2`). Notate differenze nei due file scritti?

**Soluzione**

Cambia il contenuto sia di `cipher2` che il valore del tag in `tag2`. Confrontando il tag posso accorgermi che qualcosa è stato modificato anche se non posso sapere se è stato modificato solo `plain` o anche `aad`.

Provate ad usare adesso le funzionalità di verifica dello script `aes-gcm.py`. Usate i file originali e `aad`, `ciphertext` e `tag2`. Ricevete dei messaggi di errore?

**Soluzione**

Ricevo il messaggio di errore `InvalidTag`

Adesso cambiamo il file contentente gli associated data e salvatelo in `aad2`. Eseguite quindi di nuovo lo script in decifratura/verifica con `aad2`, `cipher` e `tag`. In questo caso, lo script si accorge di differenze? Quali file sono cambiati? Provate inoltre a ricalcolare tag e ciphertext con lo script fornito (e salvare in file differenti es. `tag3`, `ciphertext3`). Notate delle differenze rispetto ai file originali? Avete ricevuto messaggi d'errore?

**Soluzione**

In questo caso cambia solo il valore del tag in `tag`.

Come ultimo aspetto di questo esercizio, è interessante valutare le prestazioni di queste procedure di AEAD e confrontarle con le prestazioni degli algoritmi di cifratura e calcolo del digest già valutate nell'esercitazione 2 (esercizi 1.3 e 2.4).

Usate gli stessi file da 100 B, 10 kB, 1 MB e 100 MB creati durante la precedente esercitazione o ricreateli con `rand` e valutate lo *user time* eseguendo questi comandi:

```
openssl rand -out file 100000000
time aes-gcm.py -e file associated cipher tag -K 123 -iv 432
```

A vostro parere conviene usare AEAD? (OK, stiamo confrontando OpenSSL con librerie di Python, tuttavia potete trarre delle conclusioni...)

**Soluzione**

Facendo un po' di esperimenti e mediando i tempi, si nota, soprattutto con i file di grosse dimensioni, che AEAD richiede poco più del tempo necessario a cifrare con AES in CBC. Invece, cifrare con AES in modalità CBC e calcolare poi un HMAC con con SHA256 richiede circa il doppio del tempo. In pratica, usando AEAD abbiamo quasi raddoppiato il throughput.

## 1.3 Firma digitale

Supponete che Alice sia ora interessata a proteggere l'integrità dei messaggi inviati a Bob con la crittografia asimmetrica. Alice possiede una coppia di chiavi RSA. Usate quella dell'esercitazione 2 oppure createla con i seguenti comandi:

```
openssl genrsa -out rsa.key.alice 2048
openssl rsa -in rsa.key.alice -pubout -out rsa.pubkey.alice
```

Supponete che Alice voglia apporre una firma ad un file scaricato da internet, ad esempio per garantire a Bob che lei ne ha verificato la correttezza formale. Supponiamo di voler applicare la firma a questo file[1]:

```
wget https://cacr.uwaterloo.ca/hac/about/chap11.pdf
```

Progettate una procedura che permetta ad Alice di firmare il file `chap11.pdf` e a Bob di verificare la firma in ricezione. Poi scrivete la sequenza di operazioni completa e corretta (usando `dgst` e `pkeyutl`):

**Soluzione**

- Alice→Bob: *rsa.pubkey.alice* (in modo sicuro)
- Alice esegue

```
openssl dgst -sha256 -binary chap11.pdf > chap11.pdf.sha256
```

- Alice esegue

```
openssl pkeyutl -sign -in chap11.pdf.sha256 -inkey rsa.key.alice -out
chap11.pdf.sha256.RSA.alice
```

- Alice → Bob: `chap11.pdf`, `chap11.pdf.sha256.RSA.alice`
- Bob esegue

```
openssl dgst -sha256 -binary chap11.pdf > chap11.pdf.sha256.bob
```

- Bob esegue

```
openssl pkeyutl -verify -in chap11.pdf.sha256.bob -sigfile
chap11.pdf.sha256.RSA.alice -inkey rsa.pubkey.alice -pubin
```

Oltre al comando `pkeyutl`, potete usare direttamente `dgst` per eseguire operazioni di firma digitale e verifica (è questo il modo consigliato nella pratica). Ad esempio, potete costruire una firma digitale del file `chap11.pdf` con:

```
openssl dgst -sha256 -sign rsa.key.alice -out chap11.pdf.sha256.RSA.alice chap11.pdf
```

Scrivete di seguito la riga di comando per verificare la firma:

**Soluzione**

Il comando da usare per verificare la firma generata con il comando sopra è il seguente:

```
openssl dgst -sha256 -verify rsa.pubkey.alice -signature
chap11.pdf.sha256.RSA.alice chap11.pdf
```

Quali differenze notate nell'uso dei comandi `pkeyutl` e `dgst`?

**Soluzione**

`dgst` automatizza tutte le operazioni sia quelle di creazione sia quelle di verifica. Inoltre `dgst` applica anche un padding (più difficile da vedere, però è scritto nel man!)

Cosa succede se Chuck modifica uno dei diversi blocchi di dati inviati da Alice a Bob? Provate con un editor esadecimale a modificare prima solo il messaggio poi solo la firma. Cosa succede invece se Chuck prova a sostituire la chiave pubblica di Alice con la sua?

[1] In alternativa, se non riuscite a scaricare questo file potete usare `/etc/apache2/apache2.conf`

**Soluzione**

Si verificano i seguenti casi:

- se Chuck altera il messaggio

```
cp chap11.pdf chap11.new.pdf
printf 'X' | dd of=chap11.new.pdf bs=1 count=1 conv=notrunc
openssl dgst -sha256 -verify rsa.pubkey.alice -signature
chap11.pdf.sha256.RSA.alice chap11.new.pdf
```

  → firma non valida (notate che `atril` apre lo stesso il file senza problemi e non ce ne saremmo mai accorti!).

- se Chuck altera la firma

```
printf '\x3C' | dd of=chap11.pdf.sha256.RSA.alice.fake bs=1 count=1
conv=notrunc
openssl dgst -sha256 -verify rsa.pubkey.alice -signature
chap11.pdf.sha256.RSA.alice.fake chap11.new.pdf
```

  → firma non valida.

- se Chuck sostituisce la chiave pubblica → MitM, non è possibile risolvere questo problema solo con la matematica su cui si basa la crittografia, ma serve della burocrazia. In ogni caso, in questo esercizio avevamo supposto che Alice avrebbe inviato a Bob la sua chiave pubblica in modo sicuro.

## 2 Applicazioni della crittografia asimmetrica: chiavi di messaggio

Supponete adesso che Alice e Bob vogliano scambiare generici messaggi cifrati, ma non abbiano una chiave simmetrica condivisa. Riuscite a progettare e mettere in pratica una soluzione basata sugli strumenti crittografici visti finora che permetta ad Alice di inviare un file cifrato a Bob, e a Bob di decifrarlo correttamente?

Supponete che Alice proceda in questo modo:

- crea[2] o scarica un file da almeno 1 MB, ad esempio

```
wget https://cacr.uwaterloo.ca/hac/about/chap12.pdf
```

- esegue

```
echo chiave > aeskey
openssl enc -aes-128-cbc -in chap12.pdf -out chap12.pdf.enc -kfile aeskey -iv 0
openssl pkeyutl -encrypt -in aeskey -inkey rsa.key.alice -out aeskey.rsa
```

- invia a Bob i tre file seguenti:

```
1. chap12.pdf.enc
2. aeskey.rsa
3. rsa.key.alice
```

Quali errori ha commesso l'inesperta Alice? (ce ne sono almeno due!)

[2] Potete usare il comando `rand` per generare un file abbastanza grande.

NOTA

Controllate con attenzione i comandi eseguiti da Alice e i file inviati a Bob.

**Soluzione**

Gli errori sono i seguenti:

1. ha cifrato la chiave simmetrica con la propria chiave pubblica (quindi la chiave simmetrica la può leggere solo chi possiede la sua chiave privata);
2. ha dovuto condividere la sua chiave privata (per ovviare all'errore precedente);
3. non ha comunicato l'IV.

Invece, avrebbe dovuto chiedere a Bob la sua chiave pubblica, ottenerla in maniera sicura e cifrare `aeskey` con quella.

Scrivete la sequenza di operazioni completa e corretta che Alice e Bob avrebbero dovuto seguire:

**Soluzione**

- Bob (B) → Alice (A): *rsa.pubkey*
- A: `echo "chiave" > aeskey`
- A: `openssl enc -aes-128-cbc -in rsaplain2 -out msg.enc -kfile aeskey`
- A: `openssl pkeyutl -encrypt -in aeskey -pubin -inkey rsa.pubkey -out aeskey.enc`
- A→B: *msg.enc*, *aeskey.enc*
- B: `openssl pkeyutl -decrypt -inkey` *rsa.key* `-in` *aeskey.enc* `-out` *aeskey*
- B: `openssl enc -d -aes-128-cbc -in` *msg.enc* `-kfile` *aeskey*

Quale attacco può ancora compiere Chuck per intercettare il messaggio scambiato? Come potreste risolvere il problema?

**Soluzione**

Il problema è sempre l'attacco MitM. La soluzione è scambiarsi le chiavi pubbliche in maniera sicura, OOB o usando certificati emessi da autorità fidate.

Quale altro algoritmo asimmetrico potrebbero usare Alice e Bob al posto di RSA per scambiarsi la chiave AES? Quali modifiche bisogna apportare alla sequenza di operazioni? Quale attacco è possibile con questo algoritmo simmetrico alternativo a RSA? Come evitare tale attacco?

**Soluzione**

Anche l'algoritmo di Diffie-Hellman permette di scambiarsi una chiave simmetrica. Lo scambio usando Diffie-Hellman deve essere svolto all'inizio, poi il messaggio deve essere cifrato con la chiave scambiata con Diffie-Hellman. Anche in questo caso esiste il rischio di MitM.

# 3 Infrastrutture a chiave pubblica

Lo scopo di questa parte dell'esercitazione è permettere di sperimentare le procedure di richiesta di certificati a chiave pubblica X.509 destinati ad utenti finali e/o server applicativi e dall'altro di acquisire una maggiore familiarità nell'analisi dei diversi formati con cui può essere rappresentato un certificato X.509.

## 3.1 La Certification Authority (CA)

Come prima cosa, create una CA di prova chiamata "Test CA" con lo script Perl `CA.pl`:

```
/usr/lib/ssl/misc/CA.pl -newca
```

**ATTENZIONE**

La tentazione di premere "Invio" ripetutamente o scrivere valori a caso per ognuno dei campi richiesti è molto forte. Tuttavia vi consigliamo di scrivere con cura questi dati, infatti una CA non emette certificati per *subject* il cui DN non è "figlio" del suo DN. Annotate quindi qui i seguenti campi:

```
countryName =
stateOrProvinceName =
organizationName =
organizationalUnitName =
```

che vengono prodotti in output dopo la creazione della CA.

Riuscite a identificare i passi seguiti dalla procedura di creazione della nuova CA?

**Soluzione**

1. creazione della coppia di chiavi
2. raccolta dei valori dei campi (country, state, organization, ecc.)
3. firma del certificato (self-signed)

Al termine, la procedura avrà creato nella directory di lavoro una struttura di directory come in Figura 1.

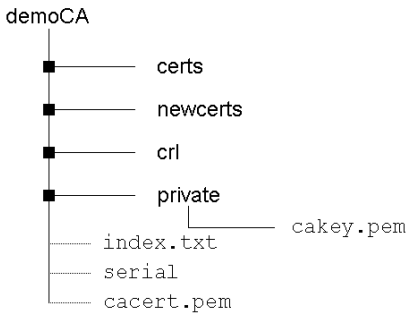



Figura 1: Struttura della CA di prova.

La chiave privata della CA è contenuta nel file `cakey.pem`. Il certificato della CA è contenuto nel file `cacert.pem`: con che chiave è firmato questo certificato? Tre importanti file accessori sono `serial` (contiene il numero di

serie del prossimo certificato da emettere), `index.txt` (contiene l'elenco dei certificati emessi ed il relativo stato) e `crlnumber` indica il numero dell'ultima CRL emessa.

Perché la CA deve tener traccia dei certificati emessi?

**Soluzione**

La CA tiene traccia dei certificati emessi (tramite il file `index.txt`) per la gestione della revoca. Permette inoltre di rilevare se la chiave privata della CA è stata compromessa e qualcun altro sta emettendo certificati a suo nome.

## 3.2 Richiesta ed emissione di certificati digitali

Adesso, Alice preparerà una richiesta di certificato da sottomettere a Carl (il personaggio che svolgerà il ruolo di CA).

Scrivete il comando OpenSSL per generare una nuova richiesta di certificato per una coppia di chiavi RSA da 2048 bit. Inserite le informazioni richieste rispettando quanto annotato in precedenza.

NOTA

Potete creare la coppia di chiavi RSA come fatto nella precedente esercitazione (`genrsa`) oppure usare direttamente il comando `req` per generare una nuova coppia di chiavi.

**Soluzione**

Per creare una nuova richiesta di certificato si possono usare i seguenti comandi:

```
openssl req -newkey rsa:2048 -pubkey -keyout alice.rsakey.pem -out
alice.certreq.pem
```

se non si ha già una coppia di chiavi RSA, oppure

```
openssl req -new -key alice.rsakey.pem -out alice.certreq.pem
```

per richiedere un certificato associato alla chiave RSA memorizzata in `alice.rsakey.pem`.

Prima di procedere, assicuratevi che la chiave privata di Alice sia salvata nel file `alice.rsakey.pem` e la richiesta di certificato in `alice.certreq.pem`. Esaminate ora la richiesta di certificato appena prodotta:

```
openssl req -in alice.certreq.pem -text -noout
```

Quale formato è usato per le richieste?

**Soluzione**

PKCS 10

Identificate natura e scopo delle diverse informazioni presenti: cosa rappresenta l'ultimo campo? A cosa serve?

**Soluzione**

È la firma applicata ai dati della richiesta PKCS#10. Serve a verificare l'integrità della richiesta e a provare il possesso della chiave privata corrispondente alla chiave pubblica presentata nella richiesta.

Non resta che sottomettere la richiesta alla CA: Alice invia[3] il file `alice.certreq.pem` a Carl. Scrivete il comando OpenSSL che Carl deve eseguire per elaborare la richiesta di Alice, emettere un nuovo certificato, e salvarlo in `alice.x509cert.pem` usando il comando `ca`:

[3] O semplicemente immaginate che lo faccia, non serve perdere tempo a scambiarsi file sulla rete.

**Soluzione**

```
openssl ca -in alice.certreq.pem -out alice.x509cert.pem
```

Quali sono stati i cambiamenti nella base dati della "Test CA" di Carl?

**Soluzione**

Il file `index.txt` è stato aggiornato aggiungendo informazioni sul certificato appena emesso ed il valore salvato nel file `serial` è stato incrementato di uno.

Carl può ora inviare[4] il certificato a Alice che può esaminare il certificato appena ricevuto usando il comando `x509` di OpenSSL:

```
openssl x509 -in alice.x509cert.pem -text -noout
```

Ora, scrivete il comando OpenSSL che Alice dovrà eseguire per verificare la validità del certificato ricevuto con il comando `verify`:

**Soluzione**

```
cat path_to_demoCA demoCA/cacert.pem > trustedCA.alice.pem
openssl verify -CAfile trustedCA.alice.pem alice.x509cert.pem
```

Di quale dati aggiuntivi ha bisogno Alice per verificare correttamente la validità del certificato?

**Soluzione**

Alice dovrà anche passare ad OpenSSL la lista delle CA che lei ritiene fidate e che non non siano già nella lista di quelle ritenute trusted da OpenSSL stesso. Tale lista deve essere passata come file unico che concateni tutti i certificati di cui si fida. In questo caso, dato che l'unica CA che abbiamo è demoCA, basterebbe anche solo passare `demoCA/cacert.pem`.

### 3.3 Revoca di certificati

Scrivete il comando OpenSSL che Carl può eseguire per revocare il certificato emesso ad Alice nel precedente esercizio

**Soluzione**

```
openssl ca -revoke demoCA/newcerts/serial_cert_alice.pem
```

NOTA

Carl non ha accesso diretto al certificato che in questa esercitazione immaginiamo usi Alice. Deve prendere le sue copie dei certificati memorizzate in `demoCA/newcerts`

Quali sono stati i cambiamenti nella base dati della "Test CA" di Carl?

**Soluzione**

In `index.txt` la riga corrispondente al certificato di Alice adesso ha 'R' (revoked) invece di 'V' (valid).

[4] O come prima, immaginate che lo faccia.

Adesso scrivete il comando per generare una nuova Certificate Revocation List (CRL) e salvarla nel file `crl.pem`:

**Soluzione**

Il comando è il seguente:

```
openssl ca -gencrl -out crl.pem
```

Carl può ora pubblicare la sua CRL. Supponiamo che Bob abbia scaricato in qualche modo la CRL di Carl. A questo punto Bob può esaminarla con il comando:

```
openssl crl -in crl.pem -text -noout
```

Provate ad intuire la funzione dei diversi campi contenuti nella CRL. Quando sarà emessa la prossima CRL? Cosa indica il campo "Revocation Date"?

NOTA

Confrontate con quanto visto a lezione a proposito delle CRL.

**Soluzione**

La prossima CRL sarà emessa secondo quanto indicato nel campo "Next Update" e avrà il campo "CRL Number" incrementato di uno.
Il campo "Revocation Date" indica la data di revoca del certificato da parte della CA. Il certificato può essere non più valido da **prima** della data di revoca. Se questo momento è noto può essere espresso aggiungendo il campo facoltativo "Invalidity Date".

Infine, scrivete il comando per permettere a Bob di verificare la CRL emessa da Carl:

**Soluzione**

```
cat path_to_demoCA/demoCA/cacert.pem > trustedCA.bob.pem
openssl crl -in crl.pem -CAfile trustedCA.bob.pem
```

# 4 Approfondimenti ed esercizi opzionali

## Applicazioni della crittografia asimmetrica

### 4.1 Autenticazione

Provate ad usare le primitive crittografiche RSA (comando `pkeyutl`) per realizzare un protocollo di autenticazione a sfida.
Lavorate in coppia (Alice e Bob) e procedete nel seguente modo:

- Alice invia a Bob la propria chiave pubblica `rsa.pubkey.alice` (potete fingere di avere Alice e Bob sullo stesso PC, in tal caso non serve inviarvi alcun file, basta scegliere i file giusti);
- Bob genera una stringa casuale nel file `random` e la cifra con la chiave pubblica di Alice, creando il file `challenge`

  **Soluzione**

  Bob:

  ```
  openssl rand -out random 20
  openssl pkeyutl -encrypt -pubin -inkey rsa.pubkey.alice -in random
  -out challenge
  ```

- Bob invia o rende disponibile sul file system ad Alice il file `challenge`;
- Alice decifra la sfida nel file `response` che invia (o rende disponibile) a Bob;

  **Soluzione**

  Alice:

  ```
  openssl pkeyutl -decrypt -inkey rsa.key.alice -in challenge -out
  response
  ```

- Bob verifica che la risposta alla sfida sia corretta

  ```
  cmp response random
  ```

Assumendo che le chiavi siano autentiche, a quale rischio è soggetto il protocollo che abbiamo

realizzato? Come si potrebbe ovviare? (almeno due soluzioni!)

**Soluzione**

In teoria, il rischio è che la sfida sia in realtà il digest di qualche documento/messaggio e Bob riesca così ad estorcere una firma digitale di Alice.
Come ovviare a questo problema? Nella pratica questi attacchi non sono possibili grazie ai formati di firma. Una firma digitale perché sia legalmente valida deve essere apposta su particolari strutture dati. Come principio generale, non bisogna spedire direttamente la risposta ma usarla in qualche verifica di conoscenza (es. autenticazione fatta con TLS).
Alice potrebbe:

- calcolare il digest della risposta `response` e inviare questo, oppure
- Alice può generare un altro numero casuale, concatenare il `challenge` a questo numero al momento del calcolo dell'hash ed inviare a Bob il digest ed il numero casuale da lei generato, oppure, più semplicemente,
- basta non accettare sfide lunghe quanto l'output degli algoritmi di hash (es. 128 bit, 160 bit, 256 bit, ...).

# Servizi basati su certificati digitali

## 4.2 Validazione di messaggi firmati

Per questo esercizio servono tre personaggi: Alice, Bob e Carl coi seguenti ruoli:

- Carl è la Certification Authority;
- Alice richiede un certificato a Carl per poter firmare i messaggi da inviare a Bob;
- Bob riceve i messaggi da Alice e ne effettua verifica;
- Chuck è un cracker, che però non svolge azioni attive per questo esercizio.

Supponiamo che Carl abbia già creato una CA valida come visto nell'esercizio al paragrafo 3.1. Inoltre supponiamo che Alice abbia un certificato X.509 emesso da Carl in corso di validità e non revocato.

NOTA

Nel caso non aveste già CA e certificati prodotti negli esercizi nei paragrafi 3.1-3.2 potete creare la CA, la richiesta ed emettete il certificato di Alice con i seguenti comandi:

```
/usr/lib/ssl/misc/CA.pl -newca
openssl req -newkey rsa:2048 -out alice.req.pem -keyout
alice.rsakey.pem
openssl ca -in alice.req.pem -out alice.x509cert.pem
```

Immaginate che Alice debba firmare quattro messaggi (`msg1`-`msg4`) ed effettui le seguenti operazioni (presentate in ordine temporale):

1. Alice usa la sua chiave privata (la cui parte pubblica è presente nel certificato) per firmare il file `msg1` con il seguente comando:

```
openssl dgst -sign alice.rsakey.pem -out msg1.firma msg1
```

2. Alice si accorge di non aver apposto la “marca temporale”, aggiunge quindi in coda al file `msg2` l’ora di firma con il seguente comando:

   ```
   date >> msg2
   ```

   ed immediatamente dopo usa la sua chiave privata (la cui parte pubblica è presente nel certificato) per firmarlo con il seguente comando:

   ```
   openssl dgst -sign alice.rsakey.pem -out msg2.firma msg2
   ```

3. il cracker Chuck riesce ad ottenere la chiave privata di Alice `alice.rsakey.pem`

4. Alice aggiunge in coda al file `msg3` l’ora di firma con il seguente comando:

   ```
   date >> msg3
   ```

   ed immediatamente dopo usa la sua chiave privata (la cui parte pubblica è presente nel certificato) per firmarlo

   ```
   openssl dgst -sign alice.rsakey.pem -out msg3.firma msg3
   ```

5. Alice si accorge che dei cracker sono riusciti ad entrare nel suo computer ed hanno copiato la sua chiave privata;

6. Alice avverte Carl e spiega che la sua chiave è stata probabilmente rubata prima che firmasse il file `msg3`, e comunica a Carl il tempo in cui lei ritiene abbiano preso la chiave nel formato `YYYYMMDDHHMMSSZ` (es. un secondo prima della marca temporale in `msg3`) visibile con:

   ```
   cat msg3
   ```

7. Carl revoca il certificato di Alice

   ```
   openssl ca -revoke demoCA/newcerts/serial_cert_alice.pem -crl_compromise
   YYYYMMDDHHMMSSZ
   ```

8. Alice aggiunge in coda al file `msg4` l’ora di firma

   ```
   date >> msg4
   ```

   ed immediatamente dopo usa la sua chiave privata (la cui parte pubblica è presente nel certificato) per firmarlo

   ```
   openssl dgst -sign alice.rsakey.pem -out msg4.firma msg4
   ```

9. Carl ed emette la CRL con il seguente comando:

   ```
   openssl ca -gencrl -out crl.carl.pem
   ```

Bob, che non conosce la sequenza di operazioni appena effettuate, riceve i file `msg{1,2,3,4}` e `msg{1,2,3,4}.firma` di Alice e deve decidere quali firme sono valide, supponendo che Bob abbia il certificato di Alice (`alice.x509cert.pem`) e che Carl sia una CA fidata per Bob (fatto includendo il certificato di Carl nel `trustedCA.bob.pem`).
Supponiamo che Bob usi la seguente procedura per verificare la validità delle firme:

1. verifica che la firma sia stata apposta con la chiave privata corrispondente a quella pubblica presente nel certificato che Alice le ha inviato;

2. verifica che la firma sia stata apposta prima che il certificato di Alice fosse revocato.

Quali operazioni l'inesperto Bob ha dimenticato nel verificare la firma di `msg1`? (minimo due operazioni!!) Modificate la procedura in modo che sia corretta e completa:

**Soluzione**

L'algoritmo completo e corretto per la verifica è il seguente:

1. identificare le CA fidate (OK, anche se il certificato di Carl non è stato scambiato in modo sicuro);
2. verificare la firma sul messaggio usando la chiave pubblica inclusa nel certificato di Alice (OK);
3. verificare la firma della CA sul certificato di Alice mediante il comando (MANCA);

   ```
   openssl verify -CAfile cacert.pem user_cert.pem
   ```

4. verificare la firma della CA sula CRL (MANCA)
5. verificare che il certificato non sia stato revocato ed che la firma sul messaggio sia stata creata prima della revoca del certificato (di Alice). (OK)

Scrivete la sequenza di comandi OpenSSL necessari ad implementare tale scenario.

NOTA

Il comando `dgst` non accetta in input chiavi pubbliche memorizzate all'interno di un certificato. È necessario passare proprio la chiave pubblica che però potete estrarre dal certificato con il seguente comando

```
openssl x509 -in alice.x509cert.pem -pubkey -noout > alice.pubkey.pem
```

**Soluzione**

```
openssl dgst -verify alice.pubkey.pem -signature msg1.firma msg1
openssl dgst -verify alice.pubkey.pem -signature msg2.firma msg2
openssl dgst -verify alice.pubkey.pem -signature msg3.firma msg3
openssl dgst -verify alice.pubkey.pem -signature msg4.firma msg4
```

```
openssl crl -in crl.carl.pem -CAfile demoCA/cacert.pem -noout
```

Quale è lo stato del certificato (revocato o non revocato) che Bob dovrà assegnare?
Completate la Tabella sottostante con i seguenti valori: V (se lo considera valido) and I (per invalido) e motivate la risposta.

| Messaggio | Tempo firma | Stato | Motivazione |
|---|---|---|---|
| msg1 | | | |
| msg2 | | | |
| msg3 | | | |
| msg4 | | | |

**Soluzione**

Assumendo che tutte le firme sono corrette dal punto di vista crittografico, così come la CRL e la scelta delle CA fidate, le decisioni si basano sulle informazioni temporali:

- `msg1.firma` non è valida perché manca la marca temporale, non è possibile quindi stabilire se `msg1` è stato firmato quando il certificato era ancora valido.
- `msg2.firma` è valida, infatti `msg2` è stato firmato quando il certificato era ancora valido;
- `msg3.firma` potrebbe essere valida, sta a Bob decidere cosa fare. Infatti, la revoca ufficiale (Revocation Date) del certificato di Alice nella CRL riporta una data posteriore alla data di firma. Purtroppo però, l'Invalidity Date riporta che secondo l'utente proprietario della chiave (cioè Alice) la chiave era stata compromessa prima dell'emissione della firma su `msg2`;
- `msg4.firma` non è valida, infatti è stata apposta quando il certificato era già stato revocato.

Quale tecnologia/protocollo avrebbe potuto usare Alice per apporre la marca temporale in maniera più affidabile mediante una Trusted Third Party?

**Soluzione**

Time Stamping e Time Stamping Authority

Se Alice inviasse il file `trusted_ca.pem` a Bob insieme agli altri messaggi, Bob dovrebbe fidarsi del suo contenuto?

**Soluzione**

No, perché un attaccante potrebbe creare la sua CA falsa. Il verificatore deve ottenere il certificato della CA con un metodo out-of-band e configurarla come trusted.

E se Alice inviasse a Bob anche la CRL (emessa da Carl) insieme agli altri messaggi, Bob potrebbe fidarsi?

**Soluzione**

Sì, perché tutti i messaggi e la CRL sono firmati, quindi possono essere verificati con trusted `CA.pem`

## Mettendo tutto assieme

### 4.3 Progettazione di protocolli

Considerando quello che avete appreso sulla crittografia simmetrica e asimmetrica, affrontate i seguenti progetti.

### 4.3.1 Protezione di messaggio

Supponete che Alice sia in possesso di una chiave pubblica RSA di Bob e la voglia usare per inviarle, via mail, un file proteggendone confidenzialità e integrità. Identificate i diversi blocchi di dati che Alice deve creare, definite un formato per imbustare i diversi blocchi di dati in un'unica mail, e create la busta. Suggerimenti:

- per il formato di imbustamento potete ispirarvi al formato PEM, lo stesso usato da OpenSSL ad esempio per salvare le chiavi RSA. PEM usa dei separatori testuali e rappresenta i dati binari in forma testuale attraverso la codifica `base64`. La codifica `base64` può essere eseguita con il comando `enc` di OpenSSL.
- quali blocchi di dati devono essere cifrati?
- quali blocchi di dati devono essere firmati?

**Soluzione**

I blocchi sono:

  - messaggio
  - eventuale chiave simmetrica, nel caso si scelga di usare la crittografia simmetrica visto che quella asimmetrica impone dei limiti sulla lunghezza del messaggio
  - digest del messaggio e dell'eventuale chiave simmetrica

Devono essere cifrati il blocco del messaggio (con la chiave simmetrica se si sceglie la cifratura simmetrica, altrimenti con la chiave asimmetrica pubblica del destinatario) e l'eventuale blocco della chiave simmetrica (con la chiave asimmetrica pubblica del destinatario).
Il digest deve essere firmato (che equivale a cifrarlo con la chiave asimmetrica privata del mittente).
NOTA: è utile aggiungere anche un blocco con il certificato della chiave pubblica del mittente da usare per la verifica della firma. Il mittente potrebbe avere più chiavi. Il destinatario deve poter verificare l'associazione utente-chiave pubblica presso una CA di cui si fida.

- ci sono differenze in termini di proprietà di sicurezza tra (1) cifrare il messaggio e poi firmare il messaggio cifrato, oppure (2) firmare il messaggio in chiaro e poi cifrare messaggio più firma?

**Soluzione**

Con l'opzione (1) decifro solo dopo averne verificato l'integrità, evitando attacchi noti come "oracolo del padding" http://en.wikipedia.org/wiki/Padding_oracle_attack, inoltre l'integrità può essere verificata anche da altri dispositivi "nel mezzo", diversi dal destinatario, es. relay, proxy.
Con l'opzione (2) ho il vantaggio di una maggiore privacy, perché lascia meno informazioni sul contenuto interno (è infatti quello usato da S/MIME).

### 4.3.2 Protezione di canale

Sempre supponendo che Alice sia in possesso di una chiave pubblica RSA di Bob, progettate ora un protocollo che permetta a Alice e Bob di scambiare messaggi attraverso un canale che garantisca confidenzialità e integrità.

Alice e Bob hanno bisogno di confidenzialità quando si scambiano i dati (cioè il canale deve essere confidenziale):

- quanti dati dovranno scambiarsi?
- i metodi visti (a lezione, in laboratorio), sono tutti compatibili con le vostre ipotesi sulla quantità dei dati?

Tra le applicazioni della crittografia, scrivete quella che vi sembra più appropriata:

**Soluzione**

La crittografia simmetrica è l'unico modo possibile per garantire la confidenzialità dei dati che di solito saranno più di quanti se ne possano cifrare usando la crittografia asimmetrica per limiti degli algoritmi (es. la lunghezza in byte del modulo meno 11 byte per RSA) o per ragioni di efficienza (almeno due ordini di grandezza).

Data la soluzione scelta in precedenza:

- quale problema rimane aperto?
- quale informazione in possesso di Alice può essere usata per risolverlo?

Scrivete di seguito il metodo che pensate sia più opportuno:

**Soluzione**

Rimane il problema di scambiarsi la chiave. Per questo si possono usare algoritmi asimmetrici, Diffie-Hellman o scambio di chiavi cifrate con RSA.

Alice e Bob hanno bisogno della garanzia che i dati scambiati siano integri:

- quali metodi hanno a disposizione?
- si può fare di meglio? cioè, si riesce ad aggiungere anche l'autenticazione dei dati?

Scrivete il metodo che pensate sia più opportuno:

**Soluzione**

Autenticazione simmetrica mediante algoritmi di hash (es. keyed-digest o HMAC).

A questo punto mettere insieme le risposte e provate a progettare un metodo per rendere sicuro il canale.

**Soluzione**

Una soluzione veramente sicura è molto difficile da progettare. Un esempio concreto è rappresentato dal protocollo SSL/TLS.

## 4.4 Formati PKCS#7

Per favorire l'interoperabilità di applicazioni basate su certificati digitali sono utilizzati diversi formati standard:

1. PKCS#7 - formato per buste sicure: in particolare permette di creare buste cifrate, firmate o cifrate e firmate che possono contenere dati (cifrati), chiavi simmetriche (cifrate), firme digitali, catene di certificati e CRL. Questo formato è usato per importare catene di certificati di CA all'interno di IE, Chrome, Firefox e altri browser. Ulteriori dettagli su questo formato sono disponibili nei seguenti documenti:
   - RFC-2315 che presenta il PKCS#7 v.1.5;
   - RFC-5652 che presenta il Cryptographic Message Syntax (CMS), estensione del PKCS#7 v.1.5 mantenuta dall'IETF.
2. PKCS#7 "degenerated mode" - formato derivato dal tipo PKCS#7 SignedData contenente unicamente catene di certificati e eventualmente CRL. Questo formato è usato per importare catene di certificati di CA all'interno di un browser (es. Firefox) o di un sistema operativo (es. Internet Options / Content di Windows).
3. PKCS#12 - formato derivato dal tipo PKCS#7 EncryptedData contenente una chiave privata, il corrispondente certificato e il certificato della CA emittente (tutto è cifrato a partire da una password). È usato per importare una coppia di chiavi e relativo certificato all'interno di un browser (es. Firefox) o di un sistema operativo (es. Internet Options / Content di Windows). Ulteriori informazioni sono disponibili presso l'RFC-7292.

I precedenti formati sono solitamente codificati in due modi:

1. DER - rappresentazione binaria; Windows usa le estensioni `.cer` o `.der` per file contenenti direttamente un singolo certificato, l'estensione `.p7b` per un file PKCS#7, l'estensione `.p12` per file PKCS#12. S/MIME usa le estensioni `.p7m` e `.p7s` per buste PKCS#7, rispettivamente cifrate e firmate.
2. PEM - codifica base64 della rappresentazione binaria in DER. Rappresentazione standard usata dai comandi OpenSSL e supportata da tutte le applicazioni più diffuse. L'estensione `.pem` è quella più comunemente usata. Per esempio, un certificato digitale codificato PEM è contenuto tra i delimitatori:

   ```
   ------BEGIN CERTIFICATE-----
   ------END CERTIFICATE-----
   ```

   Windows usa l'estensione `.cer` per identificare questo tipo di file quando contiene un singolo certificato.

OpenSSL offre una serie di comandi per la manipolazione dei precedenti formati e conversione tra essi.

### Conversione tra formati

Potete usare i seguenti comandi OpenSSL per convertire un certificato da DER a PEM e vice versa:

- convertire un certificato da formato DER a PEM:

  ```
  openssl x509 -inform DER -in file.der -outform PEM -out file.pem
  ```

- convertire un certificato da formato PEM a DER:

  ```
  openssl x509 -in file.pem -outform DER -out file.der
  ```

- convertire un file che contiene un lista di certificati in formato PEM (inclusi delimitatori) in una struttura PKCS#7:

```
openssl crl2pkcs7 -nocrl -certfile file.pem -outform PEM
      -out file.pkcs7
```

Ad esempio per convertire in formato DER il certificato di alice creato nell'esercizio potete usare il seguente comando:

```
openssl x509 -in alice.x509cert.pem -outform DER -out alice.x509cert.der
```

Per riconvertirlo in PEM invece, potete eseguire

```
openssl x509 -inform DER -in alice.x509cert.der -outform PEM -out
alice.x509cert.2.der
```

Potete infine visualizzare i campi di un certificato in formato DER con il comando OpenSSL per:

```
openssl x509 -inform DER -in usercert.der -text
```

### Buste PKCS#12

Per creare una busta PKCS#12 contenente il certificato creato nell'esercizio 3.2, la corrispondente chiave privata, e il certificato della CA che ha emesso il certificato potete eseguire il seguente comando:

```
openssl pkcs12 -export -in alice.x509cert.pem -inkey alice.rsakey.pem -name
my_certificate -certfile demoCA/cacert.pem -out alice.x509cert.p12
```

### Buste PKCS#7

Come detto in precedenza il formato PKCS#7 supporta diversi tipi di imbustamento. La sintassi generale di una busta PKCS#7 è definita in notazione ASN.1 e come radice specifica un singolo elemento chiamato `ContentInfo` strutturato come segue:

```
ContentInfo ::= SEQUENCE {
contentType ContentType,
content [0] EXPLICIT ANY DEFINED BY contentType OPTIONAL }

contentType ::= OBJECT IDENTIFIER
```

I campi di `ContentInfo` hanno il seguente significato:

- `contentType` è il tipo di dati, espresso attraverso un `OBJECT IDENTIFIER` cioè un sequenza di numeri interi assegnata in modo univoco da un'autorità. Lo standard PKCS#7 definisce sei tipi base:
  - `data` è il tipo di dato più semplice ed indica una mera sequenza di byte la cui interpretazione è lasciata alla specifica applicazione;
  - `signedData` è il tipo di dato da impiegare quando si ha la necessità di utilizzare firme digitali: vi è la definizione di un insieme di strutture accessorie necessarie per conservare tutte le informazioni utili del processo di firma;
  - `envelopedData` è il tipo di dato da impiegare quando le informazioni da elaborare devono essere cifrate con schema a chiave pubblica RSA;
  - `signedAndEnvelopedData` è il tipo di dato da utilizzare quando le informazioni da elaborare devono essere protette con cifratura a chiave pubblica RSA e contemporaneamente accompagnate da firma digitale;
  - `digestData` è il tipo di dato che si può utilizzare qualora le informazioni da trasportare debbano essere accompagnate da un hash calcolato sulle informazioni stesse;

  - `encryptedData` è il tipo di dato che si può utilizzare se le informazioni devono essere cifrate con uno schema crittografico simmetrico. Questo tipo non provvede alcun mezzo per gestire le chiavi in gioco che devono quindi essere manipolate per altre vie.

- `content` è il contenuto definito da `contentType`.

Ora potete creare un messaggio di testo ad esempio con e salvatelo nel file `msg`:

```
echo "Questo è un messaggio per l'imbustamento pkcs7" >> msg
```

ed imbustarlo con il seguente comando OpenSSL:

```
openssl smime -sign -in msg -out p7s.pem -outform PEM -signer
alice.x509cert.pem -inkey alice.rsakey.pem -certfile demoCA/cacert.pem
```

dove `alice.rsakey.pem` è la chiave privata usata per generare la firma, `alice.x509cert.pem` è il certificato di chi genera la firma e che viene imbustato e `cacert.pem` è il certificato della CA che ha emesso il certificato e che viene incluso come certification path.
Ora potete visualizzare la struttura ASN.1 della busta PKCS#7 appena generata con il seguente comando:

```
openssl asn1parse -in p7s.pem -i | less
```

Nell'ultima colonna è possibile vedere i nomi dei cambi, identificate i campi importanti e rispondete alle seguenti domande:

1. Quale dei sei tipi definiti sopra è stato usato per l'imbustamento?
2. Riuscite a identificare quali certificati sono allegati alla busta `p7s.pem`?
3. Cosa è contenuto nella struttura "pkcs7 data"?
4. Quali algoritmi crittografici sono stati usati?

**Soluzione**

1. Il tipo usato è signedData (pkcs7-signedData) contenente un sottocampo data (pkcs7-data)
2. E' allegato il certificato della CA e il certificato di Alice. Quest'ultimo contiene tutta la catena di verifica, dal certificato di Alice fino al certificato della CA radice, in questo caso quindi solo il certificato di Alice e quello della demo CA, perché non abbiamo utilizzato elementi intermedi.
3. E' contenuto il digest del messaggio in chiaro e il digest cifrato con RSA (firma). Non è contenuto il messaggio, questo formato è "detached".
4. E' stato usato Sha256 per il digest, poi cifrato con RSA encryption per generare la firma

Il seguente comando genera una firma analoga a generata dal comando precedente però lo memorizza in una busta di tipo diverso, cioè di tipo *non detached*:
`openssl smime -sign -nodetach -in msg -out p7snd.pem -outform PEM -signer alice.x509cert.pem -inkey alice.rsakey.pem -certfile demoCA/cacert.pem`
Con questo tipo di buste non si può vedere il contenuto del messaggio (che è codificato e memorizzato nella busta stessa), verificatelo con

```
openssl asn1parse -in p7snd.pem -i | less
```

**Soluzione**

Rispetto al primo tipo di busta c'è un campo "octet string [hex dump]" in più che è il messaggio (codificato, non cifrato, ma comunque non direttamente leggibile)

Infine, per verificare la firma apposta alla busta `p7snd.pem`

```
openssl smime -verify -in p7snd.pem -inform PEM -CAfile demoCA/cacert.pem
```

Il seguente comando OpenSSL serve invece a generare una busta di dipo Enveloped:

```
openssl smime -encrypt -aes-128-cbc -in msg -out p7e.pem -outform PEM
alice.x509cert.pem
```

Analizzate il file `p7e.pem` con il comando `asn1parse` ed identificate gli algoritmi utilizzati

```
openssl asn1parse -in p7e.pem -i | less
```

**Soluzione**

Il tipo usato è envelopedData (pkcs7-envelopedData) contenente un sottocampo data (pkcs7-data), il messaggio cifrato è in un campo "octect string [hex dump]", l'algoritmo di cifratura del messaggio è AES-128-CBC, l'algoritmo di cifratura della chiave simmetrica usata per cifrare il messaggio è RSA encryption.

Il seguente comando OpenSSL serve invece a decifrare la busta `p7e.pem` appena creata:

```
openssl smime -decrypt -in p7e.pem -inform PEM -inkey alice.rsakey.pem
```